

01 — Fundamentos Matemáticos

Los modelos de difusión son, en esencia, un truco elegante: destruir datos añadiendo ruido paso a paso, y luego aprender a revertir ese proceso.

Índice

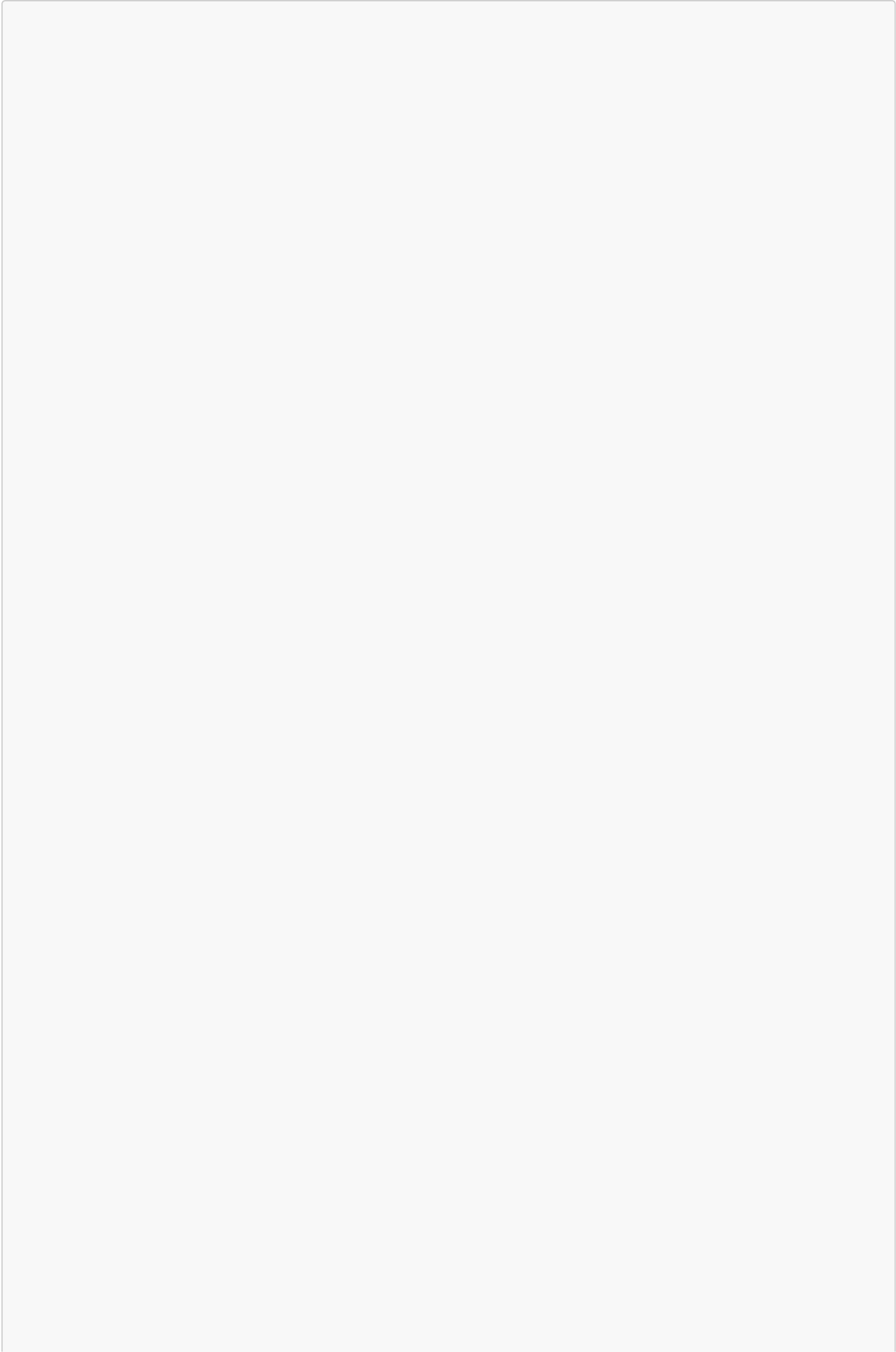
1. [El problema generativo](#)
 2. [Procesos de difusión](#)
 3. [Forward process \(destruir\)](#)
 4. [Reverse process \(reconstruir\)](#)
 5. [Noise schedules](#)
 6. [SNR — Signal-to-Noise Ratio](#)
 7. [Parametrizaciones: \$\epsilon\$, \$x_0\$, \$v\$, score](#)
 8. [ELBO — Evidence Lower Bound](#)
 9. [Reparameterization trick](#)
 10. [SDE vs ODE](#)
 11. [Diagrama conceptual completo](#)
-

1. El problema generativo

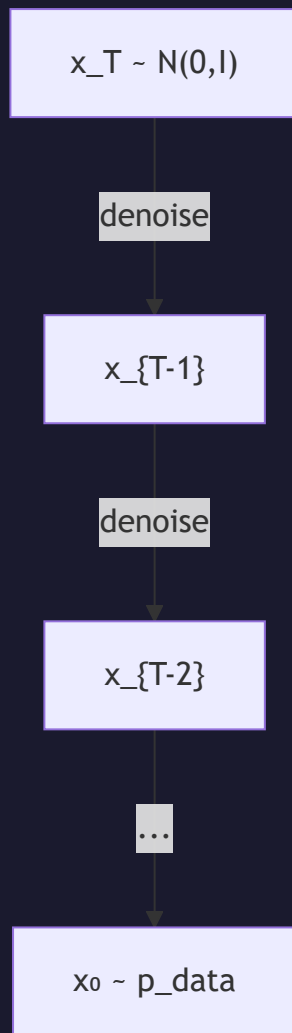
El objetivo de un modelo generativo es aprender a muestrear de una distribución de datos $p_{\text{data}}(x)$ desconocida, teniendo solo acceso a un conjunto finito de muestras (e.g., imágenes).

Enfoque de difusión: En lugar de modelar $p_{\text{data}}(x)$ directamente (como hacen los GANs o los autoregressive models), los modelos de difusión:

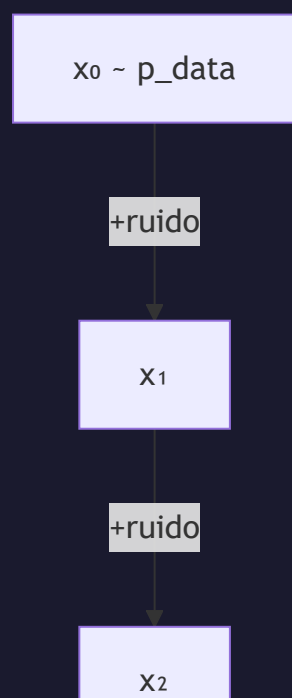
1. Definen un proceso que **destruye** la estructura de los datos gradualmente (forward)
2. Aprenden una red neuronal que **revierte** ese proceso (reverse)

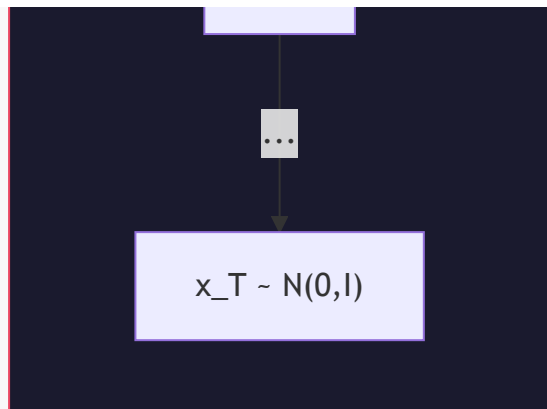


Reverse Process (aprendido)



Forward Process (fijo, no aprendido)





2. Procesos de difusión

Un **proceso de difusión** es un proceso estocástico que transforma gradualmente una distribución compleja en una distribución simple (típicamente Gaussiana).

Intuición física

El nombre "difusión" viene de la física: una gota de tinta en agua se difunde hasta que la concentración es uniforme. Los datos son la "tinta" y el ruido es el "agua".

Definición formal

Definimos una cadena de Markov con T pasos:

$$q(x_1, x_2, \dots, x_T \mid x_0) = \prod_t q(x_t \mid x_{t-1})$$

donde cada transición $q(x_t \mid x_{t-1})$ añade una cantidad controlada de ruido Gaussiano.

3. Forward process

Transición individual

En cada paso t , añadimos ruido según un schedule β_t :

$$q(x_t \mid x_{t-1}) = N(x_t; \sqrt{(1 - \beta_t)} \cdot x_{t-1}, \beta_t \cdot I)$$

Esto significa:

- **Escalar** la señal por $\sqrt{(1 - \beta_t)}$ (atenuar)
- **Añadir** ruido Gaussiano con varianza β_t

Propiedad clave: salto directo a cualquier t

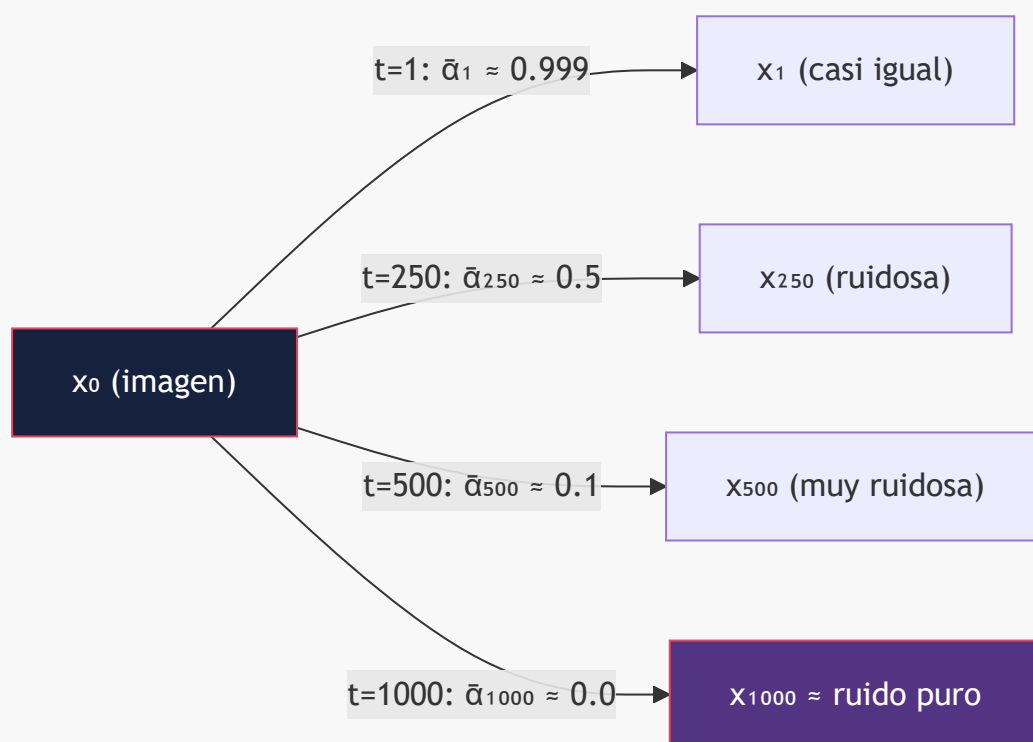
Definiendo $\alpha_t = 1 - \beta_t$ y $\bar{\alpha}_t = \prod_{s=1}^t \alpha_s$, podemos saltar directamente de x_0 a x_t :

$$q(x_t | x_0) = N(x_t; \sqrt{\bar{\alpha}_t} \cdot x_0, (1 - \bar{\alpha}_t) \cdot I)$$

O equivalentemente, usando el reparameterization trick:

$$x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{(1 - \bar{\alpha}_t)} \cdot \epsilon, \quad \epsilon \sim N(0, I)$$

Implicación práctica: No necesitamos simular los T pasos secuencialmente durante el entrenamiento. Podemos muestrear cualquier t uniformemente y calcular x_t directamente.



4. Reverse process

¿Qué necesitamos aprender?

Si conociéramos $q(x_{t-1} | x_t)$ (la distribución reversa), podríamos generar datos partiendo de ruido. Pero esta distribución **depende de la distribución completa de datos** y es intratable:

$$q(x_{t-1} | x_t) \propto q(x_t | x_{t-1}) \cdot q(x_{t-1})$$

El factor $q(x_{t-1})$ es la marginal del proceso en $t-1$, que requiere integrar sobre todo p_{data} . Ahí está la intratabilidad.

La posterior condicionada Sí tiene forma cerrada

Este es el paso que hace computable todo lo demás. Si **además de x_t condicionamos a x_0** , la marginal problemática desaparece y la posterior es Gaussiana exacta:

$$q(x_{t-1} \mid x_t, x_0) = N(x_{t-1}; \mu_t^{\sim}(x_t, x_0), \beta_t^{\sim} \cdot I)$$

con

$$\begin{aligned} \mu_t^{\sim}(x_t, x_0) &= [\sqrt{\bar{\alpha}_{t-1}} \cdot \beta_t / (1 - \bar{\alpha}_t)] \cdot x_0 + [\sqrt{\alpha_t} \cdot (1 - \bar{\alpha}_{t-1}) / (1 - \bar{\alpha}_t)] \cdot x_t \\ \beta_t^{\sim} &= [(1 - \bar{\alpha}_{t-1}) / (1 - \bar{\alpha}_t)] \cdot \beta_t \end{aligned}$$

Por qué importa: durante el entrenamiento sí conocemos x_0 (es el dato). Esta posterior es el *target* contra el que se compara $p\theta(x_{t-1} \mid x_t)$ en los términos KL del [ELBO](#). Sin ella, la §8 no sería calculable en forma cerrada.

Sustituyendo $x_0 = (x_t - \sqrt{(1-\bar{\alpha}_t)} \cdot \epsilon) / \sqrt{\bar{\alpha}_t}$ en $\mu_t^{\sim}$ se obtiene la forma que usa el sampler de DDPM:

$$\mu_t^{\sim} = (1/\sqrt{\alpha_t}) \cdot [x_t - (\beta_t / \sqrt{(1 - \bar{\alpha}_t)}) \cdot \epsilon]$$

Es decir: **predecir ϵ equivale a predecir la media de la posterior**. De ahí sale la parametrización ϵ de la §7.

Solución: aproximar con una red neuronal

Definimos:

$$p\theta(x_{t-1} \mid x_t) = N(x_{t-1}; \mu\theta(x_t, t), \Sigma\theta(x_t, t))$$

La red neuronal predice los parámetros de esta distribución Gaussiana. En DDPM, $\Sigma\theta$ se fija a $\beta_t \cdot I$ o $\beta_t^{\sim} \cdot I$ (no se aprende); Improved DDPM sí la aprende interpolando entre ambos extremos.

Resultado clave de Feller (1949)

Si el forward process es suficientemente lento (β_t pequeños) y T es grande, entonces la distribución reversa $q(x_{t-1} \mid x_t)$ es también Gaussiana. Esto justifica la parametrización Gaussiana del reverse process.

Conexión con score matching: la fórmula de Tweedie

El **score function** $\nabla_x \log q(x_t)$ es lo que permite revertir el proceso en formulación continua (ver [§10](#)). La conexión con ϵ no es una analogía: es una identidad.

Partimos de la marginal conocida $q(x_t \mid x_0) = N(\sqrt{\bar{\alpha}_t} \cdot x_0, (1 - \bar{\alpha}_t) \cdot I)$. Para una Gaussiana, el gradiente del log-densidad respecto a su argumento es:

$$\nabla_{\{x_t\}} \log q(x_t | x_0) = -(x_t - \sqrt{\bar{\alpha}_t} \cdot x_0) / (1 - \bar{\alpha}_t)$$

Y como $x_t - \sqrt{\bar{\alpha}_t} \cdot x_0 = \sqrt{(1 - \bar{\alpha}_t)} \cdot \varepsilon$, queda:

$$\nabla_{\{x_t\}} \log q(x_t | x_0) = -\varepsilon / \sqrt{(1 - \bar{\alpha}_t)}$$

La **fórmula de Tweedie** establece que el score de la marginal es la esperanza del score condicional bajo la posterior:

$$\nabla_{\{x_t\}} \log q(x_t) = E[\nabla_{\{x_t\}} \log q(x_t | x_0) | x_t] = -E[\varepsilon | x_t] / \sqrt{(1 - \bar{\alpha}_t)}$$

Y $E[\varepsilon | x_t]$ es exactamente lo que una red entrenada con MSE aprende a predecir. Por tanto:

$$s\theta(x_t, t) = -\varepsilon\theta(x_t, t) / \sqrt{(1 - \bar{\alpha}_t)}$$

Insight: entrenar un denoiser con MSE **es** hacer score matching. DDPM y los score-based models de Song & Ermon son el mismo algoritmo escrito en dos notaciones distintas.

5. Noise schedules

El schedule $\{\beta_t\}_{t=1}^T$ controla la velocidad a la que se destruye la información.

Linear schedule (DDPM original, Ho et al. 2020)

$$\beta_t = \beta_1 + (\beta_T - \beta_1) \cdot (t-1)/(T-1)$$

Con $\beta_1 = 10^{-4}$, $\beta_T = 0.02$, $T = 1000$.

Problema: La destrucción de información no es uniforme. Los primeros pasos destruyen muy poca información, los últimos destruyen demasiada. Se "desperdician" muchos timesteps.

Cosine schedule (Nichol & Dhariwal, 2021)

$$\bar{\alpha}_t = f(t) / f(0), \quad f(t) = \cos^2\left(\left(\frac{t/T + s}{1 + s}\right) \cdot \pi/2\right)$$

con **offset** $s = 0.008$. El offset no es cosmético: sin él, β_t cerca de $t=0$ tiende a cero y los primeros pasos se vuelven numéricamente degenerados. Los autores lo eligen para que $\sqrt{\beta_0}$ quede algo por debajo de la resolución de un píxel (1/127.5). En la práctica también se clipa $\beta_t \leq 0.999$ cerca de $t=T$.

Ventaja: Distribución más uniforme de la destrucción de información a lo largo de t . El SNR decrece de manera más gradual.

Comparación numérica

Valores de $\bar{\alpha}_t$ para $T = 1000$ (linear: $\beta \in [10^{-4}, 0.02]$; cosine: $s = 0.008$):

t / T	$\bar{\alpha}_t$ (linear)	$\bar{\alpha}_t$ (cosine)	Comentario
0.00	1.000	1.000	dato limpio
0.25	0.52	0.85	linear ya ha destruido la mitad de la señal
0.50	0.079	0.49	cosine llega aquí al punto de ambigüedad
0.75	0.0034	0.14	linear es casi ruido puro
1.00	$4 \cdot 10^{-5}$	~ 0	ver <i>zero-terminal-SNR</i> en §6

La lectura clave: con schedule lineal, a partir de $t \approx 0.6 \cdot T$ ya no queda casi señal, así que **el último 40 % de los timesteps aporta muy poco al entrenamiento**. El cosine reparte la destrucción de forma mucho más pareja.

Tabla de schedules

Schedule	Fórmula	Modelo(s)	Ventaja	Desventaja
Linear	β lineal en $[\beta_1, \beta_T]$	DDPM	Simple	SNR no uniforme; malgasta timesteps finales
Cosine	$\bar{\alpha}$ basado en coseno, $s=0.008$	Improved DDPM	SNR más uniforme	Ligeramente más lento en converger
Sigmoid	log-SNR lineal vía sigmoide	Chen (2023), algunos DiTs	Transición suave, un solo hiperparámetro	Requiere tuning por resolución
Scaled Linear	$\sqrt{\beta}_t$ lineal en $[\sqrt{\beta_1}, \sqrt{\beta_T}]$	Stable Diffusion (LDM)	Curva más suave al inicio que linear	Heredado de LDM, sin justificación teórica
Flow Matching	Interpolación lineal $t \in [0,1]$	FLUX, SD3	Sin schedule <i>de ruido</i> ; trayectorias rectas	Paradigma diferente; SD3 sí aplica un <i>shift</i> por resolución

Nota sobre `scaled_linear`: es lineal en la **raíz** de β , no en β . `betas = linspace( $\beta_1^{0.5}$ ,  $\beta_T^{0.5}$ ,  $T$ )2`. No tiene relación con el espacio latente — es simplemente el schedule que LDM adoptó y que Stable Diffusion 1.x/2.x heredaron.

6. SNR

Definición

El **Signal-to-Noise Ratio** en el timestep t es:

$$\text{SNR}(t) = \bar{\alpha}_t / (1 - \bar{\alpha}_t)$$

Esto mide cuánta señal original queda relativa al ruido.

Interpretación

SNR(t)	Interpretación	Lo que el modelo "ve"
$\rightarrow \infty$	Sin ruido	La imagen original
~ 1	Señal $\approx$ ruido	Mezcla ambigua
$\rightarrow 0$	Ruido puro	Gaussiana isótropa

¿Por qué importa?

- 1. **Ponderación del loss**: Diferentes timesteps contribuyen de manera diferente al aprendizaje. Los timesteps con SNR intermedio son los más informativos.
- 2. **Min-SNR weighting** (Hang et al., 2023): Ponderar el loss por $\min(\text{SNR}(t), \gamma)$ (con $\gamma = 5$) para balancear la contribución de cada timestep y evitar que los de SNR alto dominen el gradiente.
- 3. **Conexión con parametrizaciones**: La elección entre predecir ϵ , x_0 o v implícitamente cambia la ponderación del SNR en el loss (§7).

log-SNR: la parametrización invariante al schedule

Definiendo $\lambda_t = \log \text{SNR}(t) = \log(\bar{\alpha}_t / (1 - \bar{\alpha}_t))$, se obtiene una descripción del proceso **independiente de cómo se haya construido el schedule**. Kingma et al. (2021, *Variational Diffusion Models*) demuestran un resultado fuerte:

El ELBO de un modelo de difusión en tiempo continuo depende **únicamente** de los valores extremos $\lambda_{\min}$ y $\lambda_{\max}$, no de la forma de la curva λ_t entre ellos.

Esto reordena por completo la §5: linear, cosine, sigmoid y scaled-linear no son objetivos distintos, sino **reparametrizaciones del mismo objetivo** que difieren en *cuánto tiempo de cómputo* dedican a cada región de log-SNR. El schedule no cambia qué se optimiza; cambia la distribución de muestreo de t durante el entrenamiento — que es exactamente lo que SD3 manipula con su muestreo logit-normal.

Rangos típicos: $\lambda \in [-10, +10]$ cubre desde ruido esencialmente puro hasta datos casi limpios. Como $\bar{\alpha}_t = \sigma(\lambda_t)$, el log-SNR es simplemente el logit de $\bar{\alpha}$.

Zero-terminal-SNR

Un detalle que parece menor y no lo es. Con el schedule lineal de DDPM:

$$\bar{\alpha}_T \approx 4 \cdot 10^{-5} \quad \rightarrow \quad \text{SNR}(T) \approx 4 \cdot 10^{-5} \neq 0$$

x_T **no es ruido puro**: conserva una traza residual de la señal original — en concreto, del brillo medio de la imagen. Pero en inferencia se muestrea $x_T \sim N(0, I)$, que sí es ruido puro. Hay por tanto un **desajuste train/test** en el punto de partida del sampling.

Consecuencia observable (Lin et al., 2024): los modelos afectados no pueden generar imágenes muy oscuras ni muy claras. Siempre regresan hacia luminancia media, porque durante el entrenamiento nunca vieron un x_T cuyo brillo medio no delatara el de x_0 . Es la explicación de por qué a Stable Diffusion 1.5 le cuesta un fotograma verdaderamente negro.

Solución: reescalar el schedule para forzar $\bar{\alpha}_T = 0$ exactamente. Esto obliga a dos cambios acoplados:

Problema al forzar $\bar{\alpha}_T = 0$	Por qué	Solución
ϵ -prediction se vuelve indefinida en $t=T$	$x_0 = (x_t - \sqrt{(1-\bar{\alpha})}\epsilon)/\sqrt{\bar{\alpha}}$ divide por $\sqrt{\bar{\alpha}_T} = 0$	Cambiar a v-prediction , que sigue bien condicionada
El sampler debe empezar en $t=T$	Si no, nunca se corrige el desajuste	Forzar el timestep inicial del sampler

Esta es la razón de fondo por la que v-prediction dejó de ser una curiosidad teórica: **es la única de las parametrizaciones clásicas que sobrevive a SNR = 0**. Ver la nota de la [§7](#).

7. Parametrizaciones

La red neuronal puede predecir diferentes "targets". Todas son matemáticamente equivalentes, pero difieren en propiedades de optimización.

Tabla comparativa

Parametrización	La red predice	Target	Loss
ϵ -prediction	El ruido ϵ añadido	$\epsilon\theta(x_t, t) \approx \epsilon$	$\ \epsilon - \epsilon\theta(x_t, t)\ ^2$
x_0 -prediction	La imagen limpia	$x_0\theta(x_t, t) \approx x_0$	$\ x_0 - x_0\theta(x_t, t)\ ^2$
v-prediction	$v = \sqrt{\bar{\alpha}_t} \cdot \epsilon - \sqrt{(1-\bar{\alpha}_t)} \cdot x_0$	$v\theta(x_t, t) \approx v$	$\ v - v\theta(x_t, t)\ ^2$
score prediction	$\nabla_x \log q(x_t)$	$s\theta(x_t, t) \approx -\epsilon/\sqrt{(1-\bar{\alpha}_t)}$	$\ s - s\theta(x_t, t)\ ^2$

El SNR weighting implícito

Las cuatro parametrizaciones optimizan **el mismo objetivo salvo un factor de peso dependiente de t**. Reescribiendo cada loss en términos del error en ϵ (tomando ϵ -prediction como baseline uniforme):

Parametrización	Peso equivalente en espacio- ϵ	Enfatiza	Comportamiento
ϵ	1	—	Uniforme por construcción
x_0	$(1-\bar{\alpha}_t)/\bar{\alpha}_t = 1/\text{SNR}$	SNR bajo (t grande, mucho ruido)	Diverge cuando $\bar{\alpha} \rightarrow 0$

Parametrización	Peso equivalente en espacio- ϵ	Enfatiza	Comportamiento
v	$1/\bar{\alpha}_t = 1 + 1/\text{SNR}$	Balanceado	≈ 1 en SNR alto; $\approx x_0$ en SNR bajo
score	$1/(1-\bar{\alpha}_t)$	SNR alto (t pequeño, poco ruido)	Diverge cuando $\bar{\alpha} \rightarrow 1$

Derivación (para **v**, que es la menos obvia). Dado que $\hat{x}_0$ y $\hat{\epsilon}$ son consistentes con el mismo x_t :

$$\hat{v} = \sqrt{\bar{\alpha}_t} \cdot \hat{\epsilon} - \sqrt{(1-\bar{\alpha}_t)} \cdot \hat{x}_0 = \epsilon/\sqrt{\bar{\alpha}_t} - \sqrt{(1-\bar{\alpha}_t)} \cdot x_t/\sqrt{\bar{\alpha}_t}$$
$$\Rightarrow \|v - \hat{v}\|^2 = (\epsilon - \hat{\epsilon})^2/\bar{\alpha}_t \Rightarrow \|v - \hat{v}\|^2 = (1/\bar{\alpha}_t) \cdot \|\epsilon - \hat{\epsilon}\|^2$$

Análogamente $\|x_0 - \hat{x}_0\|^2 = ((1-\bar{\alpha}_t)/\bar{\alpha}_t) \cdot \|\epsilon - \hat{\epsilon}\|^2$ y $\|s - \hat{s}\|^2 = (1/(1-\bar{\alpha}_t)) \cdot \|\epsilon - \hat{\epsilon}\|^2$.

Esto es consistente con **Min-SNR** (Hang et al., 2023), que expresa los pesos en espacio- x_0 : $w = \text{SNR}$ para ϵ -pred, $w = 1$ para x_0 -pred, $w = \text{SNR} + 1$ para v -pred. Multiplicar por $1/\text{SNR}$ traduce a espacio- ϵ y reproduce la tabla de arriba.

Lectura práctica: x_0 y **score** son los dos extremos opuestos — uno se rompe en el régimen de ruido alto, el otro en el de ruido bajo. **v** es el compromiso: acotado por debajo por 1 y sin la divergencia de **score**, lo que explica su preferencia en alta resolución y en schedules con **zero-terminal-SNR**.

Relaciones entre parametrizaciones

Dado que $x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{(1 - \bar{\alpha}_t)} \cdot \epsilon$, podemos convertir entre todas:

$$x_0 = (x_t - \sqrt{(1 - \bar{\alpha}_t)} \cdot \epsilon) / \sqrt{\bar{\alpha}_t}$$
$$\epsilon = (x_t - \sqrt{\bar{\alpha}_t} \cdot x_0) / \sqrt{(1 - \bar{\alpha}_t)}$$
$$v = \sqrt{\bar{\alpha}_t} \cdot \epsilon - \sqrt{(1 - \bar{\alpha}_t)} \cdot x_0$$
$$\text{score} = -\epsilon / \sqrt{(1 - \bar{\alpha}_t)}$$

¿Cuándo usar cada una?

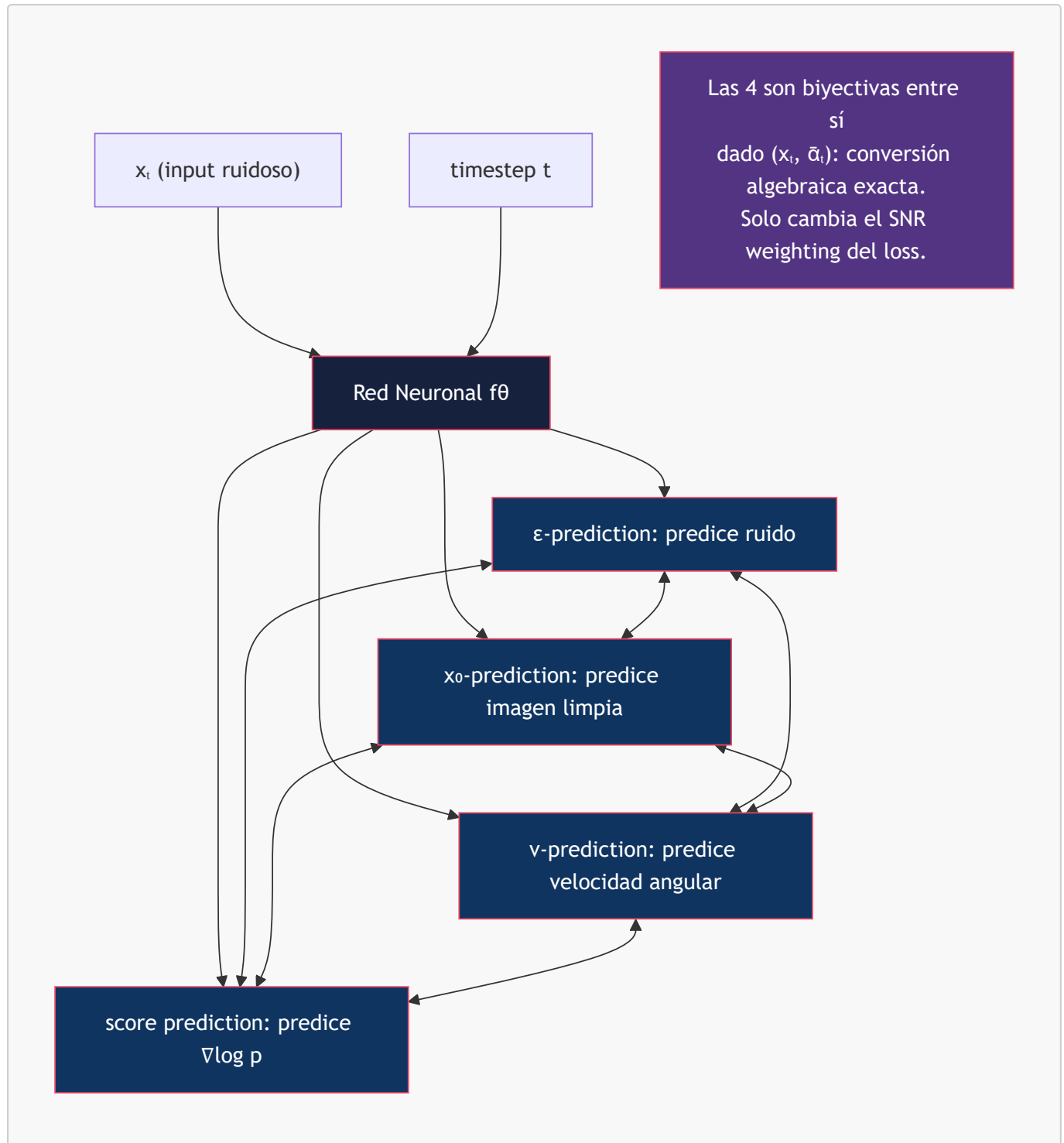
Parametrización	Cuándo conviene	Modelos que la usan
ϵ-prediction	Estándar, training estable	DDPM, Stable Diffusion 1.x, SD 2.x base (512) , SDXL 1.0
x_0-prediction	Cuando se necesita reconstrucción directa	Algunos pipelines de fine-tuning y consistency models
v-prediction	Alta resolución y schedules con zero-terminal-SNR	SD 2.x variante 768-v , Imagen Video, modelos destilados
score prediction	Formulación continua (SDE/ODE)	NCSN, Score-SDE (Song et al.)

Parametrización	Cuándo conviene	Modelos que la usan
velocity (flow)	Flow matching, trayectorias rectas	FLUX, SD3

⚠ **Corrección frecuente:** se atribuye a menudo v-prediction a SDXL. **SDXL 1.0 base usa ϵ -prediction** (`prediction_type: "epsilon"` en su configuración). El modelo de la familia Stable Diffusion que sí usa v-prediction es **SD 2.0/2.1 en la variante 768-v**.

Nota sobre velocity (flow): no es la v de v-prediction. La v de Salimans & Ho es una rotación en el plano (x_0, ϵ) sobre un schedule de difusión; la velocity de flow matching es dx/dt de una interpolación lineal $x_t = (1-t) \cdot x_0 + t \cdot \epsilon$. Coinciden en espíritu (ambas predicen una dirección de movimiento) pero no son la misma cantidad. Ver [07-flow-matching](#).

Diagrama de relaciones



8. ELBO

Derivación

El objetivo de entrenamiento se deriva del **Evidence Lower Bound** (ELBO).

Queremos maximizar la log-verosimilitud de los datos:

$$\log p_{\theta}(x_0) \geq \text{ELBO} = \mathbb{E}_q [\log p_{\theta}(x_0:T) - \log q(x_1:T \mid x_0)]$$

Que puede descomponerse en:

$$\begin{aligned} \text{ELBO} = & -\text{KL}(q(x_{-T}|x_0) \parallel p(x_{-T})) && \leftarrow \text{Prior loss } (\approx \text{constante}) \\ & - \sum_{i=2}^T \text{KL}(q(x_{i-1}|x_i, x_0) \parallel p\theta(x_{i-1}|x_i)) && \leftarrow \text{Denoising losses} \\ & + \log p\theta(x_0|x_1) && \leftarrow \text{Reconstruction loss} \end{aligned}$$

Cada término de la suma es una **KL entre dos Gaussianas**: la posterior $q(x_{i-1}|x_i, x_0)$ derivada en §4 y la predicción $p\theta(x_{i-1}|x_i)$. Ese es el motivo por el que la forma cerrada de μ_i y β_i era necesaria: sin ella este ELBO no sería computable.

Simplificación de Ho et al. (2020)

Con varianza fija ($\Sigma\theta = \beta_i \cdot I$), la KL entre dos Gaussianas de igual covarianza se reduce a la distancia entre medias:

$$\text{KL}(q \parallel p\theta) = (1 / 2\beta_i) \cdot \|\mu_i(x_i, x_0) - \mu\theta(x_i, t)\|^2 + \text{const}$$

Sustituyendo la expresión de μ_i en función de ε (también de §4) y **descartando el prefactor dependiente de t** — esa es la decisión clave de Ho et al., que equivale a reponderar el ELBO — el loss se reduce a:

$$L_{\text{simple}} = E_{t, \varepsilon} [\|\varepsilon - \varepsilon\theta(\sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{(1-\bar{\alpha}_t)} \cdot \varepsilon, t)\|^2]$$

Insight clave: El loss simplificado es simplemente un **MSE entre el ruido real y el ruido predicho**. Toda la complejidad del ELBO colapsa en un objetivo de denoising.

Matiz importante: L_{simple} ya no es el ELBO. Al descartar el prefactor $1/2\beta_i$ se cambia la ponderación relativa de los timesteps, así que L_{simple} optimiza una verosimilitud reponderada, no la log-verosimilitud. Empíricamente da mejor calidad de muestra y peor NLL. Toda la literatura de weighting posterior — Min-SNR, P2, las parametrizaciones de la §7 — consiste en revisar precisamente esta elección.

9. Reparameterization trick

Problema

Para entrenar, necesitamos muestrear $x_t \sim q(x_t | x_0)$. Pero una operación de muestreo es un nodo estocástico: no tiene derivada respecto a los parámetros de la distribución de la que se muestrea.

Solución

En lugar de muestrear $x_t \sim N(\sqrt{\bar{\alpha}_t} \cdot x_0, (1-\bar{\alpha}_t) \cdot I)$, reparametrizamos:

$$\varepsilon \sim N(0, I) \quad \leftarrow \text{Muestreo de una distribución FIJA, sin parámetros}$$

$$x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{(1 - \bar{\alpha}_t)} \cdot \epsilon \quad \leftarrow \text{Transformación determinista y diferenciable}$$

Toda la aleatoriedad queda aislada en ϵ , que no depende de nada aprendible. Lo que queda es una función diferenciable.

Qué papel juega realmente en DDPM

Conviene precisar, porque es un punto donde la exposición habitual induce a error:

- **En un VAE**, el trick es imprescindible: hay que propagar gradiente hacia los parámetros del encoder $\mu\phi(x)$, $\sigma\phi(x)$ a través del muestreo.
- **En DDPM básico**, x_0 es un dato fijo, no una cantidad aprendida. No hay ningún gradiente que necesite «fluir a través de x_0 ». El gradiente respecto a θ entra por $\epsilon\theta(x_t, t)$, no por la construcción de x_t .

Su valor real en DDPM es otro y es doble:

1. **Convierte el forward process en una operación $O(1)$** . Se puede muestrear $t \sim U\{1, T\}$ y saltar directamente a x_t sin simular los t pasos intermedios. Sin esto, el entrenamiento sería inviable.
2. **Define el target del loss**. El ϵ muestreado es la etiqueta contra la que se compara $\epsilon\theta$. La parametrización ϵ de la §7 sólo existe porque esta forma explicita ϵ como variable.

Donde el trick sí cumple su función clásica de propagación de gradiente dentro de un modelo de difusión es en **latent diffusion**, cuando el encoder del VAE se entrena conjuntamente: ahí x_0 es $z \sim q\phi(z|x)$ y el gradiente sí debe atravesar el muestreo.

10. SDE vs ODE

Los procesos de difusión pueden formularse como ecuaciones diferenciales continuas.

SDE (Stochastic Differential Equation)

$$dx = f(x, t) dt + g(t) dw$$

donde dw es un proceso de Wiener (ruido Browniano).

- **Forward SDE**: añade ruido progresivamente
- **Reverse SDE**: revierte el proceso (requiere score function)

ODE (Ordinary Differential Equation)

Song et al. (2021) mostraron que existe una **ODE determinística** (Probability Flow ODE) que genera la misma distribución marginal:

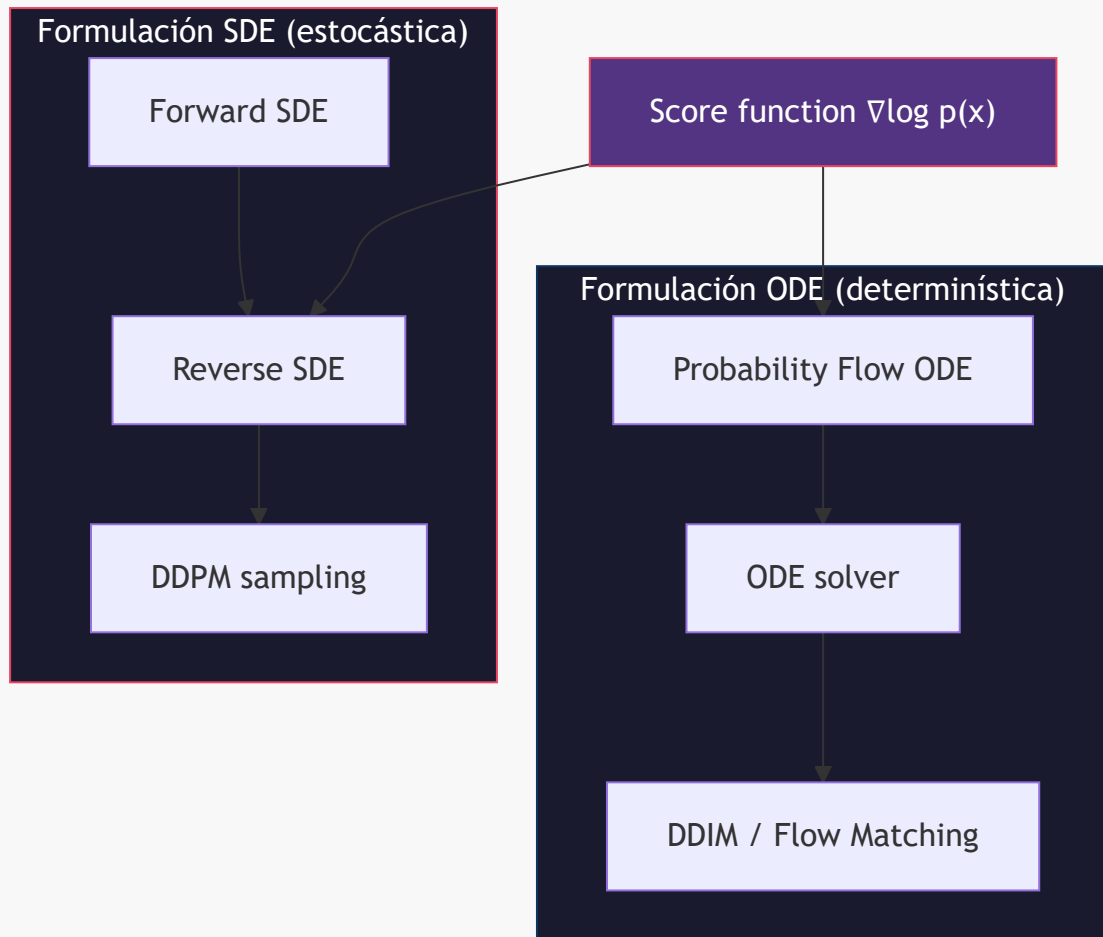
$$dx = [f(x, t) - \frac{1}{2}g(t)^2 \nabla_x \log p_t(x)] dt$$

Comparación

Aspecto	SDE (estocástico)	ODE (determinístico)
Naturaleza	Estocástico (DDPM)	Determinístico (DDIM)
Calidad muestras	Mayor diversidad	Más consistente
Número de pasos	Necesita muchos (50-1000)	Puede usar menos (10-50)
Inversión	No invertible (la traza no es recuperable)	Invertible de forma aproximada — ver nota
Interpolación	No posible directamente	Interpolación en latent space
Base teórica	SDEs	Flow Matching / ODEs

Nota sobre la invertibilidad ODE: la inversión es exacta **sólo en el límite de paso infinitesimal**. La *DDIM inversion* que se usa en la práctica discretiza la ODE y asume localmente que $\epsilon\theta(x_i, t) \approx \epsilon\theta(x_{i+1}, t+1)$, lo que introduce error de linealización que se acumula paso a paso. Con CFG alto el error crece rápido y la reconstrucción se degrada de forma visible. Precisamente por eso existe toda una familia de métodos correctivos — Null-text Inversion, EDICT, ReNoise, exact inversion vía punto fijo. Describirla como «invertible exacto» sin matizar es la fuente habitual de expectativas equivocadas en editing.

Diagrama



11. Diagrama conceptual completo



Referencias

Fundacionales

- Sohl-Dickstein, J., Weiss, E., Maheswaranathan, N., & Ganguli, S. (2015). *Deep Unsupervised Learning using Nonequilibrium Thermodynamics*. ICML. — **El paper original de difusión**; introduce el forward/reverse process y la justificación vía Feller.
- Feller, W. (1949). *On the Theory of Stochastic Processes, with Particular Reference to Applications*. Berkeley Symposium. — Resultado que justifica la gaussianidad del reverse (§4).
- Ho, J., Jain, A., & Abbeel, P. (2020). *Denoising Diffusion Probabilistic Models*. NeurIPS. — **L_simple**, schedule lineal, ϵ -prediction.

Score matching y formulación continua

- Song, Y., & Ermon, S. (2019). *Generative Modeling by Estimating Gradients of the Data Distribution*. NeurIPS. — NCSN.
- Song, Y., et al. (2021). *Score-Based Generative Modeling through Stochastic Differential Equations*. ICLR. — Unificación SDE/ODE, Probability Flow ODE (§10).
- Efron, B. (2011). *Tweedie's Formula and Selection Bias*. JASA. — Base de la identidad score $\leftrightarrow$ denoiser (§4).

Schedules, weighting y parametrizaciones

- Nichol, A. & Dhariwal, P. (2021). *Improved Denoising Diffusion Probabilistic Models*. ICML. — Cosine schedule, varianza aprendida.
- Kingma, D., Salimans, T., Poole, B., & Ho, J. (2021). *Variational Diffusion Models*. NeurIPS. — log-SNR; el ELBO continuo depende sólo de $\lambda_{\min}$, $\lambda_{\max}$ (§6).
- Salimans, T. & Ho, J. (2022). *Progressive Distillation for Fast Sampling of Diffusion Models*. ICLR. — Origen de **v-prediction**.
- Chen, T. (2023). *On the Importance of Noise Scheduling for Diffusion Models*. — Sigmoid schedule y dependencia de la resolución.
- Hang, T. et al. (2023). *Efficient Diffusion Training via Min-SNR Weighting Strategy*. ICCV.
- Lin, S., Liu, B., Li, J., & Yang, X. (2024). *Common Diffusion Noise Schedules and Sample Steps are Flawed*. WACV. — **Zero-terminal-SNR** (§6).

Flow matching

- Lipman, Y., Chen, R.T.Q., Ben-Hamu, H., Nickel, M., & Le, M. (2023). *Flow Matching for Generative Modeling*. ICLR.
- Liu, X., Gong, C., & Liu, Q. (2023). *Flow Straight and Fast: Learning to Generate and Transfer Data with Rectified Flow*. ICLR.
- Esser, P. et al. (2024). *Scaling Rectified Flow Transformers for High-Resolution Image Synthesis*. ICML. — SD3; muestreo logit-normal de t y *shift* por resolución.

Inversión (referenciado en §10)

- Mokady, R. et al. (2023). *Null-text Inversion for Editing Real Images using Guided Diffusion Models*. CVPR.
- Wallace, B., Gokul, A., & Naik, N. (2023). *EDICT: Exact Diffusion Inversion via Coupled Transformations*. CVPR.

Navegación

Siguiente en el recorrido: **02 — DDPM** *(pendiente de escribir)*

Módulos disponibles actualmente: [06 — Arquitecturas](#) · [07 — Flow Matching](#)